

```
1 # Escape-Team - Plan de développement (référence
  Codex)
2
3 Ce document sert de référence centrale pour les
  améliorations et la suite du développement du projet
  **Escape-Team**. Il décrit le périmètre fonctionnel
  , l'état du repo, les axes techniques, et une feuille
  de route priorisée.
4
5 ## Objectifs
6 - Construire un jeu en ligne **mobile-first** basé
  sur des étapes successives (A-F) et une finale.
7 - Permettre l'administration complète par un **compte
  admin** (création, pilotage, reset).
8 - Visualiser la **progression des équipes** sur un
  écran de projection.
9 - Exploiter **Stimulus** pour la logique front (
  progression, étapes, score).
10
11 ## État actuel du repo (constats)
12 - Projet Symfony avec Doctrine configuré (PostgreSQL
  attendu via `DATABASE_URL`).
13 - Sécurité minimale : provider mémoire, pas d'entité
  utilisateur persistée.
14 - Stimulus présent, mais seulement un controller de
  démo (`assets/controllers/hello_controller.js`).
15 - Aucun modèle métier (entités) dédié à Escape-Team.
16
17 ## Périmètre fonctionnel
18 ### Parcours joueur (public)
19 1. **Inscription d'équipe**
20     - Entrée par **code** et/ou **QR code**.
21     - Limite **max 10 équipes** par session.
22 2. **Salle d'attente**
23     - Attente du lancement du jeu par l'admin.
24 3. **Jeu : 6 étapes + finale**
25     - A-D : saisir une lettre (si correcte → étape
  validée).
26     - E : séquence de **5 QR codes** (unique par
  équipe). Le dernier QR valide l'étape et révèle une
  lettre.
```

```
27     - F : remise en ordre des lettres → combinaison d
    'un cryptalex numérique.
28 4. **Page finale**
29     - Affiche victoire + score.
30
31 ### Parcours admin (backoffice)
32 - **Création** d'un Escape-Team + solutions + QR
    sequences.
33 - **Liste** des Escape-Team.
34 - **Pilotage** : ouverture inscription, démarrage,
    arrêt, réinitialisation, édition, suppression.
35
36 ### Affichage public (grand écran)
37 - Page de **progression des équipes** avec scores et
    étapes.
38
39 ## Modèle de données (cible)
40 Entités proposées :
41 - `EscapeGame` : nom, statut, timestamps,
    configuration.
42 - `Team` : nom, code d'inscription, qr_token, score,
    statut.
43 - `Step` : type (A-F), solution, ordre.
44 - `TeamStepProgress` : team, step, état, timestamps,
    lettre validée.
45 - `TeamQrSequence` : team, ordre, qr_code, indice,
    validé.
46
47 Contraintes clés :
48 - Max 10 équipes par `EscapeGame`.
49 - Code d'inscription unique par équipe.
50 - Séquence QR unique par équipe (5 QR par équipe).
51
52 ## Architecture technique
53 ### Backend (Symfony)
54 - **Controllers**
55     - Public : `GameController` (inscription, attente
    , étapes, finale).
56     - Admin : `AdminEscapeController` (CRUD +
    pilotage).
57     - Scoreboard : `ScoreboardController` (projection
```

```
57 ).
58 - **API JSON**
59     - `POST /api/step/validate`
60     - `POST /api/qr/scan`
61     - `POST /api/final/check`
62     - `GET /scoreboard/data`
63 - **Sécurité**
64     - Provider Doctrine avec entité `User`.
65     - `/admin/*` protégé par `ROLE_ADMIN`.
66
67 ### Frontend (Stimulus)
68 - `game_controller.js` : validation étapes A-D.
69 - `qr_controller.js` : séquence QR + indices.
70 - `final_controller.js` : validation combinaison
    finale.
71 - `scoreboard_controller.js` : refresh périodique (
    polling).
72
73 ### Templates (Twig)
74 - `templates/game/*` : pages publiques (mobile-first
    ).
75 - `templates/admin/escape/*` : backoffice.
76 - `templates/scoreboard/index.html.twig` : grand
    écran.
77
78 ## Feuille de route (priorisée)
79 ### Phase 1 – Socle admin + sécurité
80 - Créer l'entité `User` et configurer le provider
    Doctrine.
81 - Ajouter login + protection `/admin`.
82 - Créer la base de données (migrations).
83
84 ### Phase 2 – Modèle métier + endpoints
85 - Implémenter les entités Escape-Team.
86 - Ajouter endpoints JSON pour validation des étapes.
87 - Mettre en place le scoring et les états.
88
89 ### Phase 3 – UI publique + Stimulus
90 - Construire les pages mobiles publiques (inscription
    → finale).
91 - Ajouter la logique Stimulus (validation,
```

```
91 progression, QR).
92
93 ### Phase 4 – Backoffice complet
94 - CRUD Escape-Team + pilotage (ouvrir, lancer, stop
  , reset).
95 - Gestion des solutions (lettres, QR sequence).
96
97 ### Phase 5 – Scoreboard
98 - Page grand écran + API de progression.
99 - Rafraîchissement automatique.
100
101 ## Règles UX / mobile
102 - Mobile-first, navigation simple (1 action
  principale par écran).
103 - Limiter la saisie (auto-focus, gros boutons).
104 - QR : prévoir fallback manuel (code texte) si
  caméra indisponible.
105
106 ## Tests (cible)
107 - Tests unitaires de validation (lettres, QR,
  combinaison finale).
108 - Tests d'intégration (inscription, progression,
  reset).
109
110 ## Notes techniques
111 - La configuration Doctrine est déjà prête pour
  PostgreSQL.
112 - Le frontend Stimulus est déjà câblé via `assets/
  controllers`.
113
114 ---
115
116 **Référence Codex** : ce document doit être mis à
  jour à chaque évolution majeure.
```